PharmOS

Community pharmacy, simplified.

Prototype Extension, Product Backlog, and Architectural Design

Scope

This stage extends the existing PharmOS prototype with two practical workflows, estimates and organizes the backlog, and defines a simple desktop architecture.

The scope stays intentionally small: the goal is to make a few core pharmacy workflows work well.

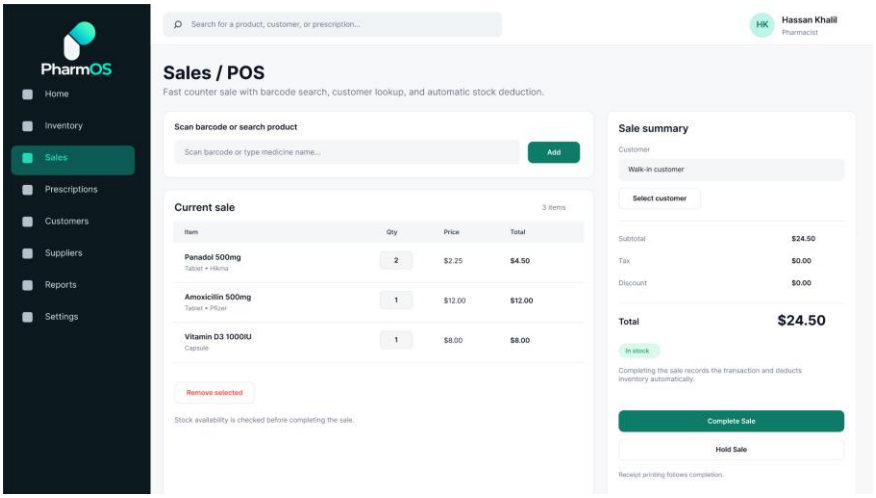
Part One - Prototype Extension

The previous prototype covered the homepage, inventory attention view, and Smart Stock recommendation.

For this stage, two workflows were added to cover frequent day-to-day pharmacy work without expanding the project too far.

1. Sales / POS

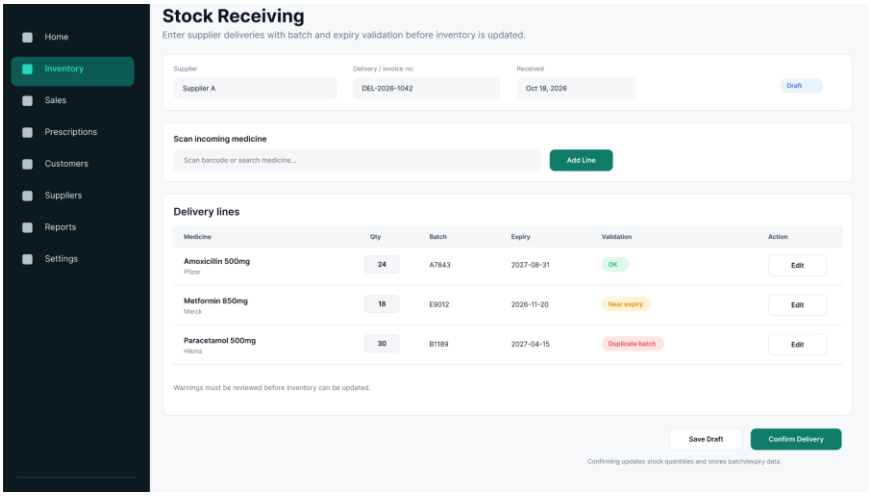
This screen demonstrates the basic counter-sale flow: scan or search for a medicine, review the current sale, optionally select a customer, and complete the transaction. Completing a sale is intended to record the transaction, deduct stock, and then print a receipt.



Figma: Open Sales / POS screen

2. Stock Receiving

This screen focuses on receiving supplier deliveries. Staff can scan a medicine, enter quantity, batch, and expiry information, and review warnings before inventory is updated. Duplicate batches and unusually close expiry dates are shown as validation warnings.



Figma: Open Stock Receiving screen

Prototype Validation

The prototype was checked against the main personas and scenarios rather than claiming a full external user study. The purpose was to verify that the screens support the intended workflow and expose the important decisions clearly.

Persona / task	What was checked	Result	Observation
Karim - rush-hour sale	Find product, build sale, see total, complete transaction	Pass	The POS keeps the sale in one screen and does not require leaving the workflow.
Rania - supplier delivery	Scan item, enter batch/expiry, review warnings, confirm delivery	Pass	The receiving screen separates data entry from confirmation and catches questionable data before stock changes.
Maya - stock planning	Open alert, inspect Smart Stock reasoning, edit quantity, create draft	Pass	The existing three-screen Smart Stock flow still keeps the final purchasing decision under staff control.

Validation takeaway: no major navigation change is needed. The main risk is implementation quality - sales and receiving must update inventory reliably, and Smart Stock must stay explainable and advisory.

Part Two - Product Backlog

1. Story point estimation

The stories were estimated relatively using Fibonacci story points (1, 2, 3, 5, 8). The estimation video emphasizes relative estimates rather than trying to convert every story directly into hours. Any story estimated above 5 points was split before entering the final backlog.

Original story	Init estimate	Split
Complete sale: record transaction, print receipt, and deduct stock	8	US-04A Record sale and deduct stock (5) US-04B Print receipt after sale (2)
Receive stock in one workflow and update inventory	8	US-14A Scan and enter delivery line (5) US-14B Confirm delivery and update inventory (3)

2. Final backlog

The final backlog contains all product stories plus a small set of necessary development activities. The complete CSV is included separately for Jira import.

ID	Type	Backlog item	Pts	Priority	Component
US-01	Story	Scan medicine into inventory	5	High	Inventory
US-02	Story	Show low-stock and expiry alerts	3	High	Inventory
US-03	Story	Add product to sale by barcode or search	3	Highest	POS
US-04A	Story	Record sale and deduct stock	5	Highest	POS
US-04B	Story	Print receipt after sale	2	Medium	POS
US-05	Story	Search customer history	3	Medium	Customer Records
US-06	Story	Link sale or prescription to customer	3	Medium	Customer Records
US-07	Story	Rank shortage-risk items	5	High	Smart Stock
US-08	Story	Explain Smart Stock recommendation	3	High	Smart Stock
US-09	Story	Keep stock suggestions advisory	2	Highest	Smart Stock
US-10	Story	Show actionable dashboard	5	High	Dashboard
US-11	Story	Open filtered inventory from alert	3	High	Dashboard
US-12	Story	Edit suggested reorder quantity	3	High	Smart Stock
US-13	Story	Review reorder draft before confirmation	5	High	Smart Stock
US-14A	Story	Scan and enter delivery line	5	High	Inventory
US-14B	Story	Confirm delivery and update inventory	3	High	Inventory
US-15	Story	Warn about duplicate batch or close expiry	3	High	Inventory
DEV-01	Task	Create local database schema		High	Platform
DEV-02	Task	Implement login and basic roles		High	Platform
DEV-03	Task	Implement backup and restore		High	Platform
DEV-04	Task	Add audit logging		Medium	Platform
DEV-05	Task	Integrate barcode scanner input		High	Hardware
DEV-06	Task	Integrate receipt printer		Medium	Hardware
DEV-07	Task	Test critical inventory and POS workflows		High	Quality

3. Jira organization

A Jira-ready CSV accompanies this document. The backlog can be imported with the columns Summary, Work Type, Description, Priority, Story Point Estimate, and Component. A simple To Do / In Progress / Done board is enough.

Part Three - Architectural Design

1. Complete story set grouped by component

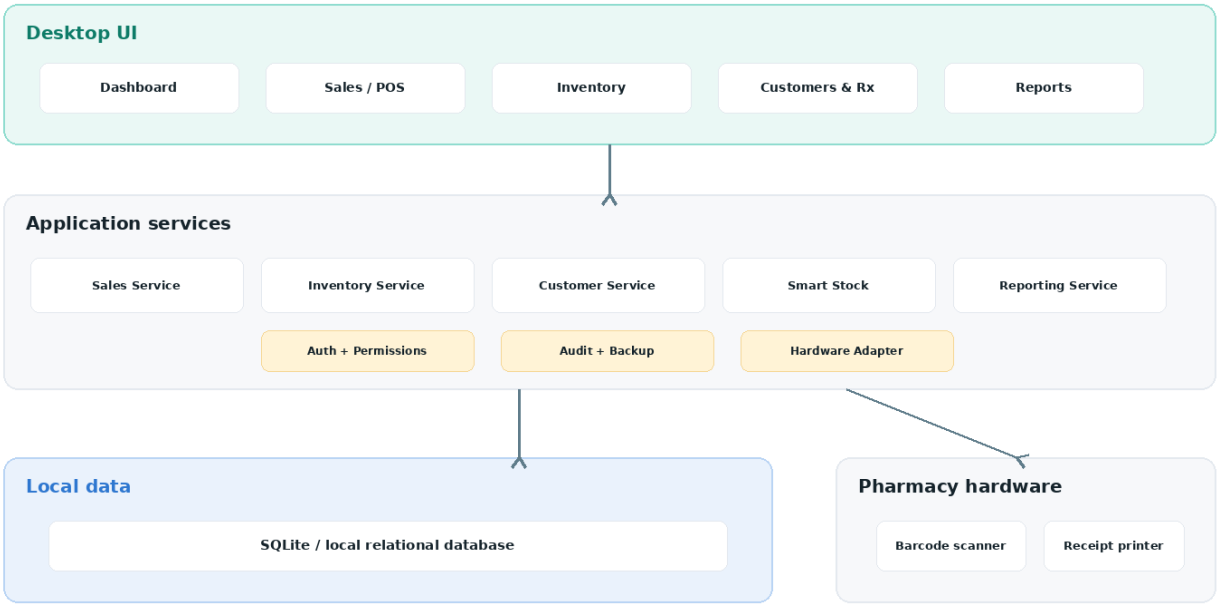
Software component	Stories	Responsibility
Inventory	US-01, -02, -14A, -14B, -15	Medicine, low-stock/expiry, stock receiving, validation, and inventory updates.
Sales / POS	US-03, -04A, -04B	Product scan/search, sale completion, stock deduction, and receipt printing.
Customer Records	US-05, US-06	Customer search, prescription/purchase history, and linking transactions.
Smart Stock	US-07, -08, -09, -12, -13	Risk ranking, explainable recommendations, editable reorder quantities, and human approval.
Dashboard / Reporting	US-10, -11	Actionable homepage information and direct navigation from alerts.

2. Non-functional requirements

Requirement	Description
NFR-1 Reliability / data integrity	A failed sale or delivery must not leave stock partially updated. Critical database updates should succeed or roll back together.
NFR-2 Performance	Barcode lookup, product search, and opening the main workflow screens should normally respond within about 2 seconds on a typical pharmacy PC.
NFR-3 Usability	Common counter tasks should stay within a few clicks and support keyboard/barcode-scanner use without deep menus.
NFR-4 Security	Staff must authenticate, and sensitive actions should respect basic user permissions.
NFR-5 Backup and recovery	The local database must be backed up and restorable without losing transaction history.
NFR-6 Offline operation	Core sales, inventory, customer, and reporting workflows should work without an internet connection.
NFR-7 Hardware compatibility	The application should work with common keyboard-style barcode scanners and a receipt printer.
NFR-8 Maintainability / auditability	Main modules should be separated, and important stock/sale changes should be logged with user and time.

3. Architecture

A simple layered desktop architecture is enough for the first PharmOS version. The UI modules call application services instead of editing the database directly. The service layer for rules such as stock deduction, delivery confirmation, Smart Stock calculations, authentication, auditing, and backups.



- Desktop UI - Dashboard, Sales/POS, Inventory, Customer Records, and Reports screens.
- Application services - Sales, Inventory, Customer, Smart Stock, and Reporting services contain the main business rules.
- Platform services - Authentication/permissions, audit logging, backup/restore, and hardware adapters support application.
- Local data - relational DB stores products, batches, sales, customers, prescriptions, suppliers, reorder drafts, and audit records.
- Hardware - Barcode scanner input and receipt printing are isolated behind the hardware adapter so the core logic does not depend on a specific device.

It is intentionally not a microservice or cloud-heavy design. PharmOS targets one small community pharmacy and needs reliable local operation more than deployment complexity. Separating the UI, services, data, and hardware is enough to keep the system testable and maintainable while staying realistic for the project.

Architecture design (image), and general text refinement (+table creation) were completed with the help of ChatGPT